Exercise 2: E-commerce Platform Search Function

Scenario:

You are working on the search functionality of an e-commerce platform. The search needs to be optimized for fast performance.

Steps:

1. Understand Asymptotic Notation:

o Explain Big O notation and how it helps in analyzing algorithms.

o Describe the best, average, and worst-case scenarios for search operations.

2. Setup:

o Create a class Product with attributes for searching, such as productId, productName, and category.

3. Implementation:

o Implement linear search and binary search algorithms.

o Store products in an array for linear search and a sorted array for binary search.

4. Analysis:

o Compare the time complexity of linear and binary search algorithms.

o Discuss which algorithm is more suitable for your platform and why.

Big O Notation

Big O notation describes how the running time or memory usage of an algorithm grows as the input size (`n`) increases.

It focuses on the algorithm's growth rate rather than the exact execution time.

For example:

- If there are 10 products, a search may take milliseconds.
- If there are 10 million products, the algorithm's efficiency becomes very important.

Big O helps us compare algorithms independently of hardware or programming language.

Best, average, and worst-case scenarios for search operations.

1. Linear Search

Linear Search checks each element one by one until the required element is found or the array ends.

Best Case – O(1)

The required product is found at the first position, so only one comparison is needed.

Example:
Searching for Product ID 101 in:

101 102 103 104 105

found immediately

Average Case – O(n)

The required product is somewhere in the middle of the array, so about half of the elements are checked before it is found.

Example:
Searching for Product ID 102:

101 102 103

Worst Case – O(n)

The required product is at the last position or not present in the array. In this case, every element must be checked.

Example:
Searching for Product ID 105:

101 102 103 104 105

or searching for Product ID 999, which is not in the array.

2. Binary Search

Binary Search works only on a sorted array. It repeatedly checks the middle element and eliminates half of the remaining elements.

Best Case – O(1)

The required product is the middle element, so it is found in the first comparison.

Example:

101 102 103 104 105 106 107

Searching for 104 finds it immediately.

Average Case – O(log n)

The required product is found after checking the middle element a few times, reducing the search space by half each step.

Example:

101 102 103 104 105 106 107

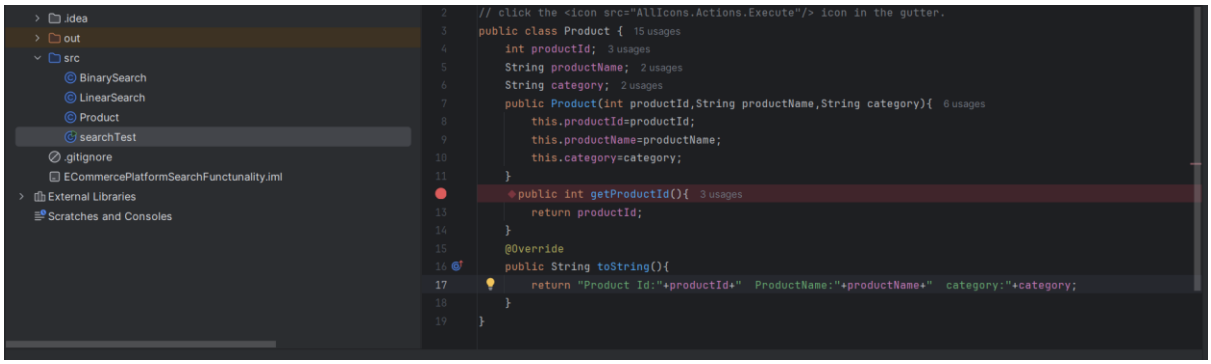
Searching for 106

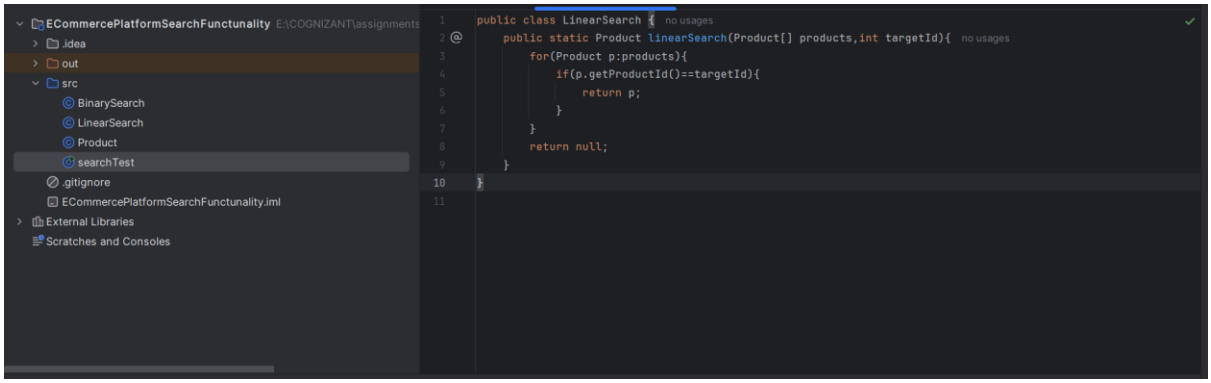
Worst Case – O(log n)

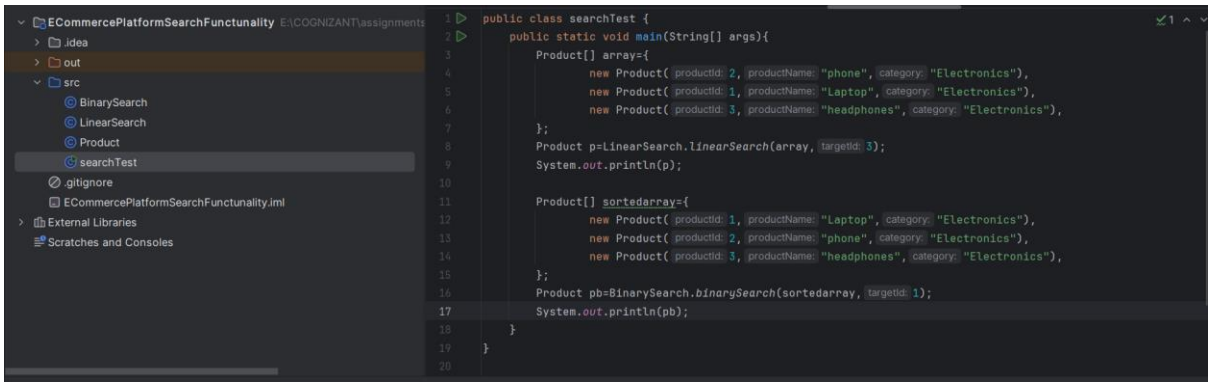
The required product is at one end of the array or is not present. Even then, Binary Search keeps reducing the search space by half until the search ends.

Example:

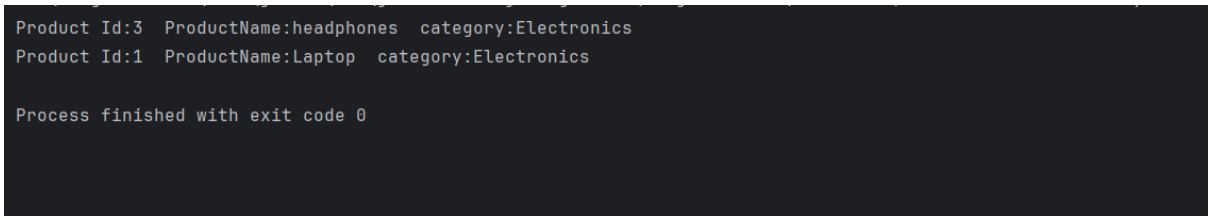
Searching for 101 or a product that is not present in the sorted array.







OUTPUT



For an e-commerce platform**,** Binary Search is generally more suitable when searching a sorted array, as it provides much faster search performance (O(log n)) compared to Linear Search (O(n)). This makes it a better choice for large product catalogs where search speed is important.